

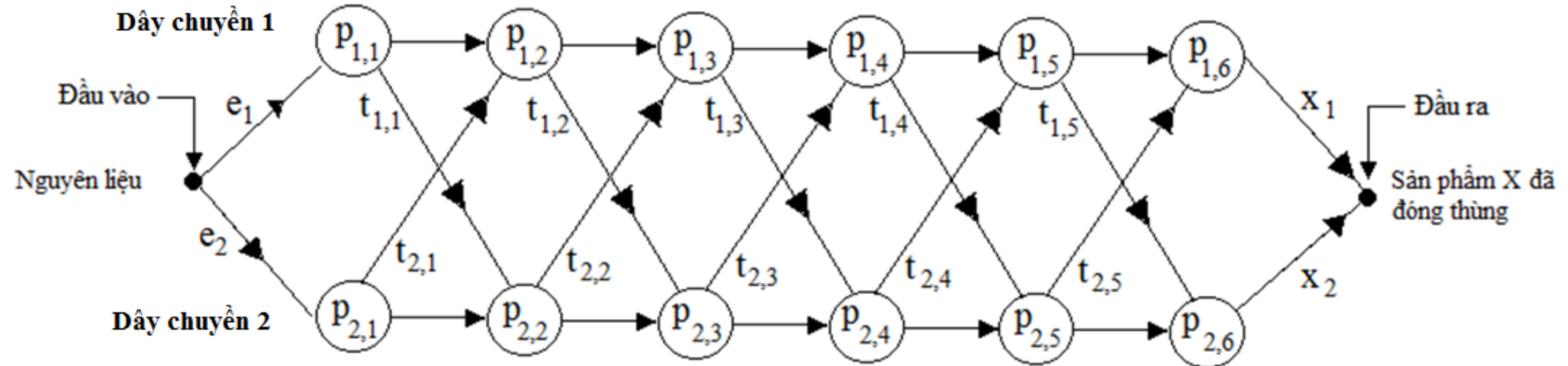
Chương 6 : Quy Hoạch Động

6.1 Bài toán 1 :

Để sản xuất một món hàng X cần thực hiện 6 công đoạn. Giả sử có 2 dây chuyền sản xuất X.

- Ký hiệu công đoạn j của dây chuyền 1 là $P_{1,j}$, thời gian là $P_{1,j}$,
- công đoạn j của dây chuyền 2 là $P_{2,j}$, thời gian là $P_{2,j}$,
- Thời gian đưa nguyên liệu vào $P_{1,1}$ (công đoạn đầu tiên của dây chuyền 1) là e_1 ,
- Thời gian đưa nguyên liệu vào $P_{2,1}$ (công đoạn đầu tiên của dây chuyền 2) là e_2 ,
- Thời gian lấy sản phẩm X từ $P_{1,6}$ (công đoạn cuối của dây chuyền 1) đóng vào thùng là x_1 ,
- Thời gian lấy sản phẩm X từ $P_{2,6}$ (công đoạn cuối của dây chuyền 2) đóng vào thùng là x_2 ,
- Thời gian chuyển sản phẩm từ $P_{i,j}$ sang $P_{i,j+1}$ là 0,
- Ta cho phép chuyển sản phẩm từ $P_{1,j}$ sang $P_{2,j+1}$, thời gian $t_{1,j}$,
- Thời gian chuyển sản phẩm từ $P_{2,j}$ sang $P_{1,j+1}$ là $t_{2,j}$.

Bài toán : Hãy tìm các công đoạn $P_{1,j}$ và $P_{2,k}$ để thời gian từ nguyên liệu đến khi X được đóng thùng là ngắn nhất.



Bài toán : Hãy tìm các công đoạn $P_{1,j}$ và $P_{2,k}$ để thời gian từ nguyên liệu đến khi X được đóng thùng là ngắn nhất.

➤ Ta có thể giải bài toán bằng cách sau :

- tìm tất cả các tập con của dây chuyền 1, một tập con như vậy ta gọi là S,
- mỗi tập con của dây chuyền 1 (S) sẽ kết hợp với các tập con thích hợp ở dây chuyền 2,
- VD : Ở dây chuyền 1 ta chọn $S = \{ P_{1,1}, P_{1,3}, P_{1,4}, P_{1,6} \}$, ở dây chuyền 2 sẽ là $P_{2,2}, P_{2,5}$.
Tính thời gian cần thực hiện cho tất cả các tập S và chọn S có thời gian ngắn nhất.

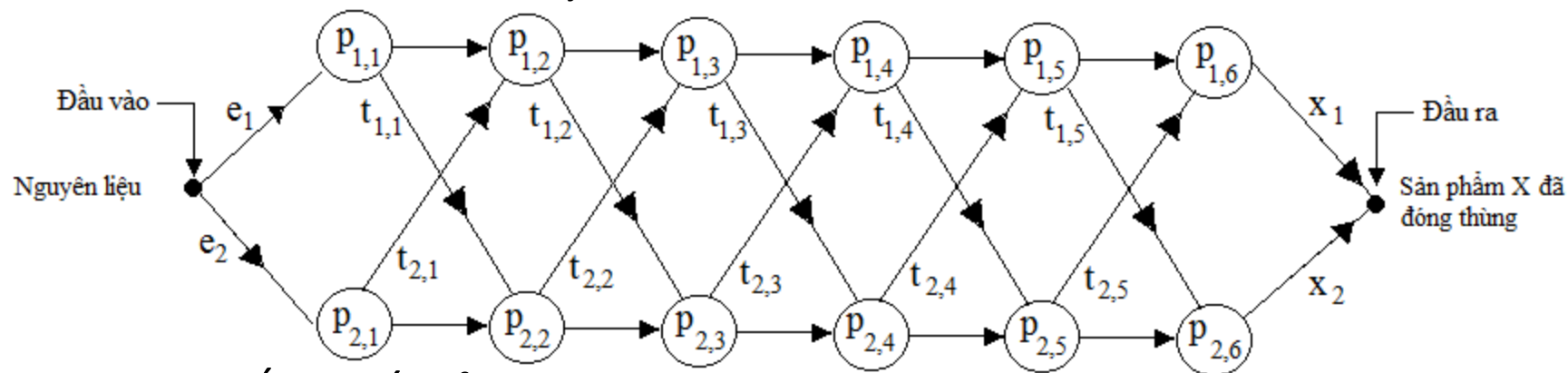
⇒ Thời gian cần thiết là 2^6 . Tổng quát là 2^n , n là số công đoạn.

⇒ Một thuật toán thực hiện với thời gian 2^n được đánh giá là không tốt.

➤ Quy hoạch động là phương pháp xây dựng thuật toán mà thời gian thực hiện thuật toán là nhỏ.

Giải bài toán : Giả sử nguyên liệu từ *đầu vào* và đi qua các công đoạn $P_{1,1}, P_{2,2}, P_{1,3}, P_{1,4}$, được gọi là *một đường đi qua công đoạn $P_{1,4}$* . Có nhiều đường đi qua công đoạn $P_{1,4}$.

Bước 1 : Tìm cấu trúc của đường đi qua $P_{i,j}$ có thời gian là ngắn nhất. *Tìm cấu trúc của lời giải tối ưu.*



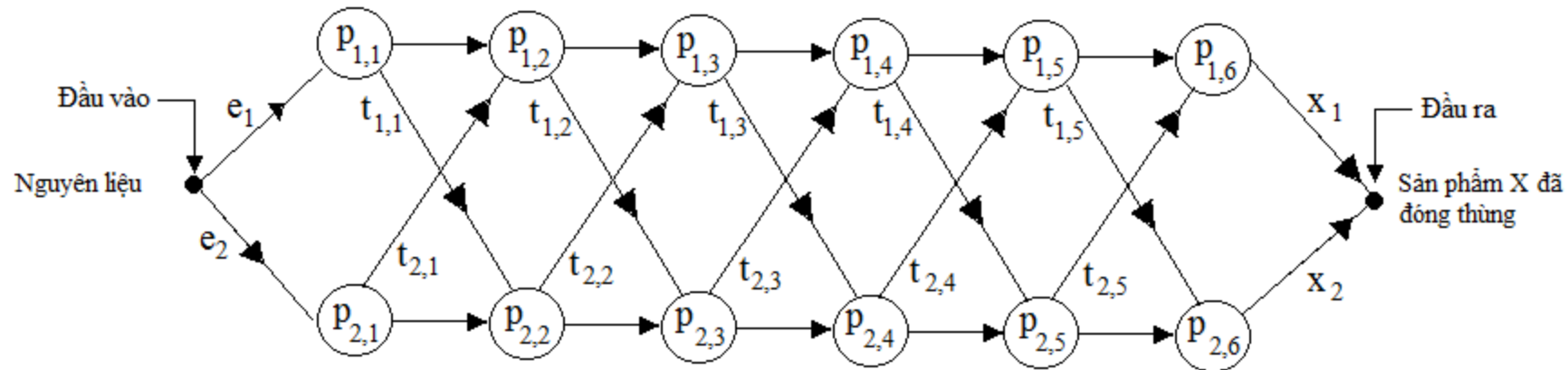
- Gọi $d[P_{1,4}]$ là thời gian ngắn nhất của một đường đi qua $P_{1,4}$. Ta có :

$$d[P_{1,4}] = \begin{cases} d[P_{1,3}] + P_{1,4} & \text{nếu } d[P_{1,3}] + P_{1,4} < d[P_{2,3}] + t_{2,3} + P_{1,4} \\ d[P_{2,3}] + t_{2,3} + P_{1,4} , & \text{nếu ngược lại} \end{cases}$$

$$\Leftrightarrow d[P_{1,4}] = \min (d[P_{1,3}] + P_{1,4} , d[P_{2,3}] + t_{2,3} + P_{1,4}).$$

$\Rightarrow$ Giá trị của lời giải ở bước j được tính từ lời giải bước j-1.

Bước 2 : Phát biểu cấu trúc của đường đi qua $P_{i,j}$ có thời gian là ngắn nhất dưới dạng đệ qui.



$$d[P_{1,4}] = \min (d[P_{1,3}] + P_{1,4} , d[P_{2,3}] + t_{2,3} + P_{1,4}).$$

$$\left[\begin{array}{l} d[P_{1,1}] = P_{1,1} + e_1, d[P_{2,1}] = P_{2,1} + e_2, \\ d[P_{1,j}] = \min(d[P_{1,j-1}] + P_{1,j} , d[P_{2,j-1}] + t_{2,j-1} + P_{1,j}), \quad j \geq 2, \\ d[P_{2,j}] = \min(d[P_{2,j-1}] + P_{2,j} , d[P_{1,j-1}] + t_{1,j-1} + P_{2,j}), \quad j \geq 2, \end{array} \right.$$

Bước 3 : Thuật toán ,

$$d[P_{1,1}] = P_{1,1} + e_1, d[P_{2,1}] = P_{2,1} + e_2,$$

$$d[P_{1,j}] = \min(d[P_{1,j-1}] + P_{1,j}, d[P_{2,j-1}] + t_{2,j-1} + P_{1,j}), \quad j \geq 2,$$

$$d[P_{2,j}] = \min(d[P_{2,j-1}] + P_{2,j}, d[P_{1,j-1}] + t_{1,j-1} + P_{2,j}), \quad j \geq 2,$$

FASTEST-WAY($P_{i,j}, t_{i,j}, e_1, e_2, x_1, x_2, n$) {

1. $f1[1] = P_{1,1} + e_1$; $f2[1] = P_{2,1} + e_2$; // $f1[1] \equiv d[P_{1,1}]$, $f2[1] \equiv d[P_{2,1}]$

2. **for** ($j = 2$; $j \leq n$; $j++$) {

4. **if** ($f1[j-1] + P_{1,j} \leq f2[j-1] + t_{2,j-1} + P_{1,j}$) // $f1[j] \equiv d[P_{1,j}]$, $f2[j] \equiv d[P_{2,j}]$

5. { $f1[j] = f1[j-1] + P_{1,j}$; $l1[j] = 1$; }

6. **else** { $f1[j] = f2[j-1] + t_{2,j-1} + P_{1,j}$; $l1[j] = 2$; }

7. **if** ($f2[j-1] + P_{2,j} \leq f1[j-1] + t_{1,j-1} + P_{2,j}$) {

8. { $f2[j] = f2[j-1] + P_{2,j}$; $l2[j] = 2$; }

9. **else** { $f2[j] = f1[j-1] + t_{1,j-1} + P_{2,j}$; $l2[j] = 1$; }

}//**end for**

10. **if** ($f1[n] + x_1 \leq f2[n] + x_2$)

11. { $fr = f1[n] + x_1$; $lr = 1$; }

12. **else** { $fr = f2[n] + x_2$; $l2 = 2$; }

} // **end FASTER**

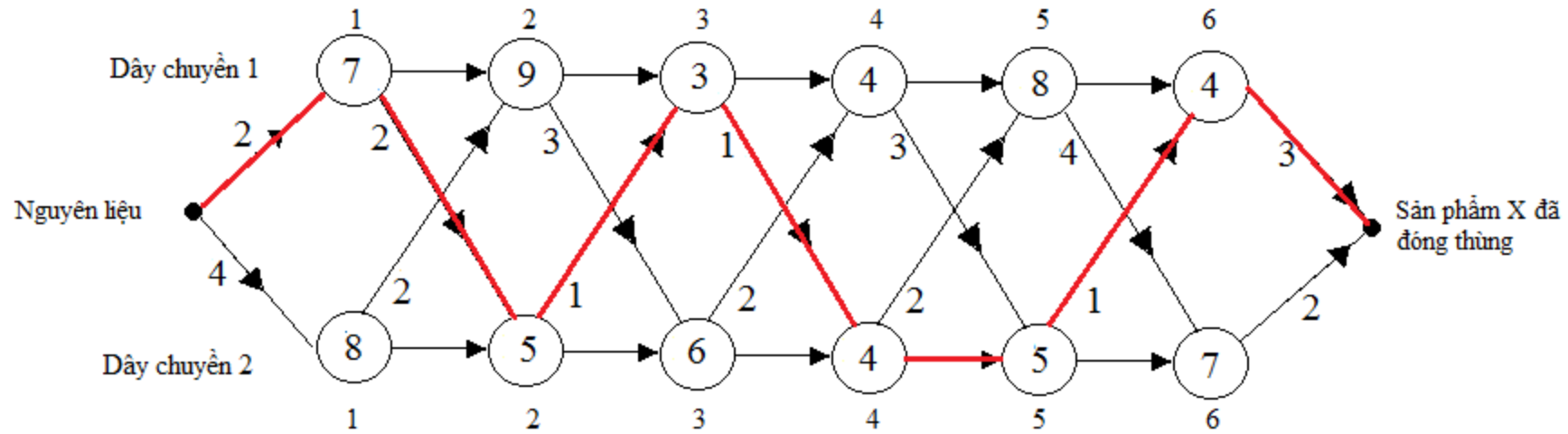
Bước 3 : Thuật toán ,

Đặt $P1[j] \equiv P_{1,j}$, $P2[j] \equiv P_{2,j}$, $t1[j] \equiv t_{1,j}$, $t2[j] \equiv t_{2,j}$,
FASTEST-WAY($P1[]$, $P2[]$, $t1[]$, $t2[]$, e_1 , e_2 , x_1 , x_2 , n)

```
{  
1.  f1[1] = P1[1] + e1 ; f2[1] = P2[1] + e2 ; // f1[1] ≡ d[P1,1], f2[1] ≡ d[P2,1]  
2.  for (j = 2 ; j <= n; j++){  
4.      if (f1[ j - 1] + P1[j] ≤ f2[ j - 1] + t2[j-1] + P1[j]) // f1[j] ≡ d[P1,j], f2[j] ≡ d[P2,j]  
5.      { f1[ j] = f1[ j - 1] + P1[j] ; l1[ j] = 1; }  
6.      else { f1[ j] = f2[ j - 1] + t2[j-1] + P1[j] ; l1[ j] = 2; }  
  
7.      if (f2[ j - 1] + P2[j] ≤ f1[ j - 1] + t1[j-1] + P2[j])  
8.      { f2[ j] = f2[ j - 1] + P2[j]; l2[ j] = 2; }  
9.      else { f2[ j] = f1[ j - 1] + t1[j-1] + P2[j] ; l2[ j] = 1; }  
    }//end for
```

```
10. if (f1[n] + x1 ≤ f2[n] + x2)  
11. { fr = f1[n] + x1; lr = 1;}  
12. else {fr = f2[n] + x2; l2 = 2;}  
    } // end FASTER
```

Ví dụ :



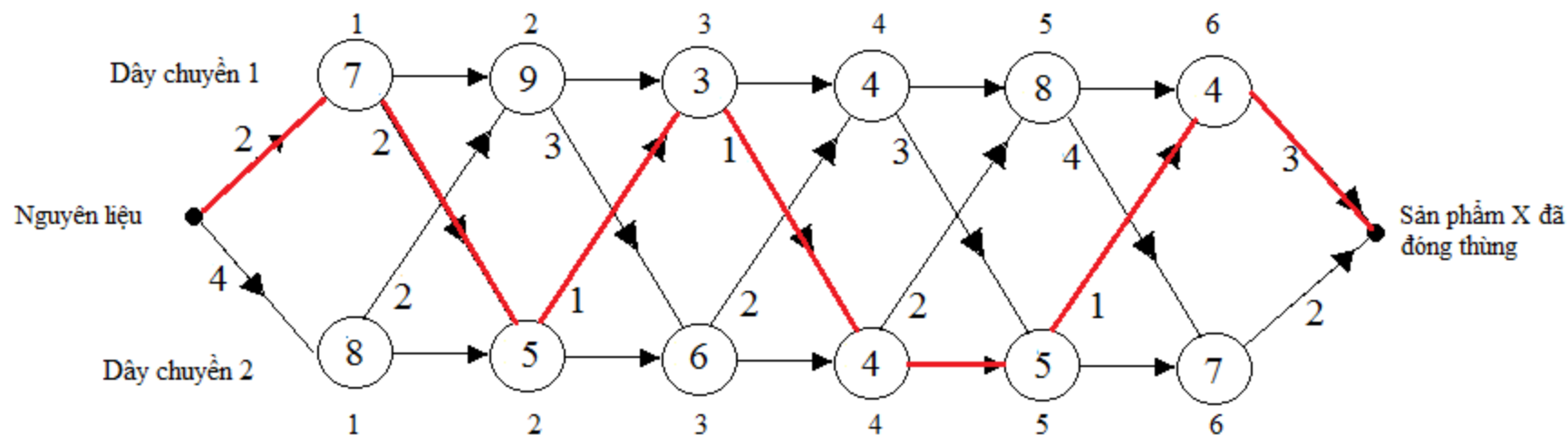
j	1	2	3	4	5	6
f1[j]	9	18	20	24	32	35
f2[j]	12	16	22	25	30	37

$$fr = 38$$

j	2	3	4	5	6
l1[j]	1	2	1	1	2
l2[j]	1	2	1	2	2

$$lr = 1$$

Ví dụ :



void main()

```
{ int P1[]={0, 7,9,3,4,8,4}, P2[]={0, 8,5,6,4,5,7}, n = 6;
```

```
int t1[]={0, 2,3,1,3,4}, t2[]={0, 2,1,2,2,1};
```

```
int e1=2, e2=4, x1=3, x2=2;
```

```
FASTEST-WAY(P1, P2, t1, t2, e1, e2, x1, x2, n);
```

```
Viết kết quả ; (BT)
```

```
}
```

6.2 Bài toán 2 : Nhân nhiều ma trận

▪ Nhân 2 ma trận :

- Cho ma trận $A = (a_{ij})$ có kích thước $p \times q$ ($i=1, \dots, p, j=1, \dots, q$), và
- ma trận $B = (b_{ij})$ có kích thước $q \times r$,
- Tích của ma trận A và B là ma trận C được định nghĩa :

$$C = (c_{ij}) = AB, \text{ có kích thước } p \times r, \quad c_{ij} = \sum_{k=1}^q a_{ik} b_{kj},$$

➤ Tích các ma trận có tính kết hợp : $(AB)C = A(BC)$, $(AB)(CD) = (A(BC))D$

▪ Thuật toán nhân 2 ma trận : $C = AB$

MATRIX-MULTIPLY(A, B, C)

```
{  
    for (i = 1 ; i <= p; i++)  
        for (j = 1 ; j <= r ; j++) {  
            C[i][ j] = 0 ;  
            for (k = 1; k <= q ; k++) C[i][ j] = C[i][ j] + A[i][ k]*B[k][ j] ; (*)  
        }  
}
```

Độ phức tạp của thuật toán là số lần thực hiện lệnh (*), ký hiệu NSM (the number of scalar multiplications).

$$NSM = pqr$$

- **Đặt dấu ngoặc hoàn toàn** : Cho một dãy ma trận $A_1, A_2, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Xét tích

$$A_1 A_2 \dots A_n \quad (1)$$

Đặt dấu ngoặc hoàn toàn cho tích (1) được định nghĩa :

- $n = 1$: đặt dấu ngoặc hoàn toàn cho (1) là A_1 ,
- $n = 2$: đặt dấu ngoặc hoàn toàn cho (1) là $(A_1 A_2)$,
- $n > 2$: đặt dấu ngoặc hoàn toàn cho (1) là (BC) ,

với B là một *đặt dấu ngoặc hoàn toàn* cho $A_1 \dots A_k$ và C là một *đặt dấu ngoặc hoàn toàn* cho $A_{k+1} \dots A_n$, k là một số nguyên $1 \leq k \leq n-1$.

Ví dụ : Xét tích $A_1 A_2 A_3 A_4$

$$\begin{aligned} (A_1(A_2(A_3 A_4))) &, B \equiv A_1 & , C \equiv (A_2(A_3 A_4)), \\ (A_1((A_2 A_3)A_4)) &, B \equiv A_1 & , C \equiv ((A_2 A_3)A_4), \\ ((A_1 A_2)(A_3 A_4)) &, B \equiv (A_1 A_2) & , C \equiv (A_3 A_4), \\ ((A_1(A_2 A_3))A_4) &, B \equiv (A_1(A_2 A_3)) & , C \equiv A_4, \\ (((A_1 A_2)A_3)A_4) &, B \equiv ((A_1 A_2)A_3) & , C \equiv A_4. \end{aligned}$$

Bài tập : $A_1(A_2 A_3 A_4)$
được đặt dấu ngoặc hoàn
toàn ?

Ý nghĩa : ?

- Bài toán : Cho một dãy 3 ma trận A_1, A_2, A_3 , lần lượt có kích thước $10 \times 100, 100 \times 5$, and 5×50 . Tính NSM khi tính tích $A_1A_2A_3$. Thực hiện tính tích theo 2 cách sau (áp dụng tính kết hợp) :

Cách 1 : $((A_1A_2)A_3)$, được đặt dấu ngoặc hoàn toàn

- Bước 1 : gọi MATRIX-MULTIPLY(A_1, A_2, B), B là kết quả , có kích thước $10 \times 5 \Rightarrow \text{NSM} = 10.100.5 = 5000$,
- Bước 2 : gọi MATRIX-MULTIPLY(B, A_3, C), $C = A_1A_2A_3$, có kích thước $10 \times 50 \Rightarrow \text{NSM} = 10.5.50 = 2500$.

$\Rightarrow \text{NSM}$ khi tính tích $A_1A_2A_3 = 5000 + 2500 = \mathbf{7500}$.

Cách 2 : $(A_1(A_2A_3))$, được đặt dấu ngoặc hoàn toàn

- Bước 1 : gọi MATRIX-MULTIPLY(A_2, A_3, B), B là kết quả , có kích thước $100 \times 50 \Rightarrow \text{NSM} = 100.5.50 = 25000$,
- Bước 2 : gọi MATRIX-MULTIPLY(A_1, B, C), $C = A_1A_2A_3$, có kích thước $10 \times 50 \Rightarrow \text{NSM} = 10.100.50 = 50000$.

$\Rightarrow \text{NSM}$ khi tính tích $A_1A_2A_3 = 50000 + 25000 = \mathbf{75000}$.

$\Rightarrow$ **Cách 1 chạy nhanh hơn 10 lần.**

- Bài toán : Cho một dãy ma trận $A_1, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Hãy tìm một *đặt dấu ngoặc hoàn toàn* cho tích $A_1 \dots A_n$ (2) để NSM của nó là bé nhất.
- Bài toán có thể được giải quyết bằng cách tìm tất cả các *đặt dấu ngoặc hoàn toàn* cho tích (2) và tính NSM cho mỗi *đặt dấu ngoặc hoàn toàn*. Tìm NSM nhỏ nhất.
- Để tìm tất cả các *đặt dấu ngoặc hoàn toàn* cần thời gian xấp xỉ 2^n .

- Giải bài toán : Cho một dãy ma trận $A_1, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Hãy tìm một đặt dấu ngoặc hoàn toàn cho tích $A_1 \dots A_n$ để NSM của nó là bé nhất.

Bước 1 : Tìm cấu trúc của *đặt dấu ngoặc hoàn toàn tối ưu* (cho tích $A_1 \dots A_n$ có NSM giá trị bé nhất). *Tìm cấu trúc của lời giải tối ưu.*

Nhận xét :

- Giả sử đã tìm được một *đặt dấu ngoặc hoàn toàn* tối ưu.
- Kết quả tìm được có dạng (BC), với B là một *đặt dấu ngoặc hoàn toàn* cho $A_1 \dots A_k$ và C là một *đặt dấu ngoặc hoàn toàn* cho $A_{k+1} \dots A_n$, k là một số nguyên $1 \leq k \leq n-1$.
- Ta có B và C cũng là tối ưu. Nghĩa là NSM của $A_1 \dots A_k$ tính theo B là bé nhất, NSM của $A_{k+1} \dots A_n$ tính theo C là bé nhất (Tại sao ? : Bài tập).

- Giải bài toán : Cho một dãy ma trận $A_1, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Hãy tìm một đặt dấu ngoặc hoàn toàn cho tích $A_1 \dots A_n$ để NSM của nó là bé nhất.

Bước 1 : Tìm cấu trúc của *đặt dấu ngoặc hoàn toàn* tối ưu (cho tích $A_1 \dots A_n$ có NSM giá trị bé nhất). *Tìm cấu trúc của lời giải tối ưu.*

- Gọi $m[i, j]$ là NSM nhỏ nhất của $A_i \dots A_j$, nhận xét tương tự như trên sẽ có *đặt dấu ngoặc hoàn toàn* tối ưu B, *đặt dấu ngoặc hoàn toàn* tối ưu C và *đặt dấu ngoặc hoàn toàn* (BC) là tối ưu cho tích $A_i \dots A_j$.
- $m[i, k]$ là NSM nhỏ nhất của $A_i \dots A_k$ tính theo B, $m[k+1, j]$ là NSM nhỏ nhất của $A_{k+1} \dots A_j$ tính theo C, $i \leq k < j$,
- $m[i, i] = 0, i = 1, \dots, n$,

$\Rightarrow$ Giá trị cần tìm là $m[1, n]$.

$\Rightarrow$ Tìm B, C tối ưu $\Rightarrow m[1, n] = m[1, k] + m[k+1, n] + p_0 p_k p_n$.

B có kích thước $p_0 \times p_k$, C có kích thước $p_k \times p_n$.

NSM của tích $A_1 \dots A_k$ và $A_{k+1} \dots A_n$

- Giải bài toán : Cho một dãy ma trận $A_1, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Hãy tìm một đặt dấu ngoặc hoàn toàn cho tích $A_1 \dots A_n$ để NSM của nó là bé nhất.

Bước 2 : Phát biểu cấu trúc của $m[1, n]$ dưới dạng đệ qui.

Tìm B, C tối ưu $\Rightarrow m[1, n] = m[1, k] + m[k+1, n] + p_0 p_k p_n \Leftrightarrow$

Tìm k có $m[1, k] + m[k+1, n] + p_0 p_k p_n$ bé nhất $\Leftrightarrow$

$$\Rightarrow m[1, n] = \min_{1 \leq k < n} \{ m[1, k] + m[k+1, n] + p_0 p_k p_n \}$$

- $m[1, k], m[k+1, n]$ được tính tương tự như $m[1, n]$. Nghĩa là

$$- m[1, k] = \min_{1 \leq p < k} \{ m[1, p] + m[p+1, k] + p_0 p_p p_k \}$$

$$- m[k+1, n] = \min_{k+1 \leq p < n} \{ m[k+1, p] + m[p+1, n] + p_{k+1} p_p p_n \}$$

- Tương tự cho $m[1, p], m[p+1, k], m[k+1, p], m[p+1, n]$

...

Tổng quát : Công thức đệ qui

$$- m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}, \quad i < j,$$

$$- m[i, i] = 0.$$

- Giải bài toán : Cho một dãy ma trận $A_1, \dots, A_n$, ma trận A_i có kích thước $p_{i-1} \times p_i$. Hãy tìm một đặt dấu ngoặc hoàn toàn cho tích $A_1 \dots A_n$ để NSM của nó là bé nhất.

Bước 2 : Phát biểu cấu trúc của $m[1, n]$ dưới dạng đệ qui.

Tổng quát : Công thức đệ qui

- $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, \quad i < j,$
- $m[i, i] = 0.$

- $$m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, \quad i < j,$$

- $$m[i, i] = 0.$$

Ví dụ :

Ma trận	Kích thước	Ma trận	Kích thước	Ma trận	Kích thước
A1	30 × 35	A3	15 × 5	A5	10 × 20
A2	35 × 15	A4	5 × 10	A6	20 × 25

Tính $m[i, i + 1], i = 1, \dots, n-1$
Tính $m[i, i + 2], i = 1, \dots, n-2$

Ma trận m

	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0		
5					0	
6						0

$$m[1, 2] = \min \{ m[1,1] + m[2, 2] + 30.35.15 \} = 15750,$$

$$m[2, 3] = \min \{ m[2,2] + m[3,3] + 35.15.5 \} = 2625,$$

$$m[3, 4] = \min \{ m[3,3] + m[4,4] + 15.5.10 \} = 750,$$

Ma trận S

k

	2	3	4	5	6
1	1				
2		2			
3			3		
4					
5					

- $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, i < j,$

- $m[i, i] = 0.$

Ví dụ :

Ma trận	Kích thước	Ma trận	Kích thước	Ma trận	Kích thước
A1	30 × 35	A3	15 × 5	A5	10 × 20
A2	35 × 15	A4	5 × 10	A6	20 × 25

Tính $m[i, i + 1], i = 1, \dots, n-1$
 Tính $m[i, i + 2], i = 1, \dots, n-2$

	1	2	3	4	5	6
1	0	15750				
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

	2	3	4	5	6
1	1				
2		2			
3			3		
4				4	
5					5

$$m[4, 5] = \min \{ m[4,4] + m[5,5] + 5.10.20 \} = 1000,$$

$$m[5, 6] = \min \{ m[5,5] + m[6,6] + 10.20.25 \} = 5000,$$

- $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, i < j,$

- $m[i, i] = 0.$

Ví dụ :

Ma trận	Kích thước	Ma trận	Kích thước	Ma trận	Kích thước
A1	30 × 35	A3	15 × 5	A5	10 × 20
A2	35 × 15	A4	5 × 10	A6	20 × 25

Tính $m[i, i + 1], i = 1, \dots, n-1$
 Tính $m[i, i + 2], i = 1, \dots, n-2$

	1	2	3	4	5	6
1	0	15750	7875			
2		0	2625			
3			0	750		
4				0	1000	
5					0	5000
6						0

	2	3	4	5	6
1	1	1			
2		2			
3			3		
4				4	
5					5

$$m[1, 3] = \min \{ m[1,1] + m[2,3] + 30.35.5 \quad (= 0 + 2625 + 5250 = 7875, k = \mathbf{1}), \\ m[1,2] + m[3,3] + 30.15.5 \quad (= 15750 + 0 + 2250 = 18000, k = 2) \} = 7875,$$

- $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, i < j, (3)$

- $m[i, i] = 0.$

Ví dụ :

Ma trận	Kích thước	Ma trận	Kích thước	Ma trận	Kích thước
A1	30 × 35	A3	15 × 5	A5	10 × 20
A2	35 × 15	A4	5 × 10	A6	20 × 25

Tính $m[i, i + 1], i = 1, \dots, n-1$
 Tính $m[i, i + 2], i = 1, \dots, n-2$
 ...
 Tính $m[1, 1+n-1]$

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

	2	3	4	5	6
1	1	1	3	3	3
2		2	3	3	3
3			3	3	3
4				4	5
5					5

$\Rightarrow$ NSM tối ưu khi tính tích $A_1A_2A_3A_4A_5A_6 = 15125.$

- $m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1}p_kp_j \}, i < j, (3)$
- $m[i, i] = 0.$

Nhận xét :

- Nếu tính $m[1, n]$ theo công thức đệ qui (3) thì thời gian xấp xỉ 2^n ,
- Lập bảng như trên thì thời gian xấp xỉ $n^3 \ll 2^n$.
- **Lập bảng như trên, bước tiếp theo , bước tính $m[i, j]$, sử dụng lại những giá trị đã tính trước đó mà cách tính đệ qui không làm được.**

Bước 3 : Thuật toán. p mảng các kích thước của các A_i

MATRIX-CHAIN-ORDER(p, m, s) // A_i có kích thước $p[i-1] \times p[i]$

```
{
1. for (i = 1 ; i <= n; i++) m[i, i] = 0;
2. for (r = 2 ; r <= n ; r++)
4.   for (i = 1 ; i <= n - r + 1; i++)
5.     {   j = i + r - 1 ;
6.         m[i, j] =  $\infty$  ;
7.   for (k = i ; k <= j - 1; k++)
8.     {
9.       q = m[i, k] + m[k + 1, j] +
10.         p[i-1]*p[k]*p[j];
11.       if (q < m[i, j])
12.         { m[i, j] = q ; s[i, j] = k ; }
13.     } // 7.
14.   } // 4.
15. } // end MATRIX-CHAIN-ORDER
// m, s : kết quả
```

Xác định cấu trúc của *đặt dấu ngoặc hoàn toàn* tối ưu.

Ma trận	Kích thước	Ma trận	Kích thước	Ma trận	Kích thước
A1	30×35	A3	15×5	A5	10×20
A2	35×15	A4	5×10	A6	20×25

	2	3	4	5	6
1	1	1	3	3	3
2		2	3	3	3
3			3	3	3
4				4	5
5					5

PrintOP(i, j) // PRINT-OPTIMAL-PARENS(s[], i, j)

```

{
1. if (i == j)
2.     printf ( "A%d", i);
   else {
3.     print f("(");
4.     PrintOP(s, i, s[i, j]);
5.     PrintOP(s, s[i, j] + 1, j);
6.     print (")");
   }
}

```

```

main()
{
  Nhập mảng p;
  MATRIX-CHAIN-ORDER(p, m, s);
  PrintOP(s, 1, 6);
  MultiplyMatrix(As); (Bài tập);
}
As : dãy các ma trận.

```


6.3 Bài toán 3 : Tìm chuỗi con chung dài nhất

▪ Định nghĩa chuỗi con :

- Cho chuỗi $X = x_1 x_2 \dots x_m = \langle x_1, x_2, \dots, x_m \rangle$, chuỗi $Z = \langle z_1, z_2, \dots, z_k \rangle$ được nói là **chuỗi con của chuỗi X** nếu tồn tại các chỉ số $i_1 < i_2 < \dots < i_k$ sao cho $x_{i_j} = z_j, j = 1, \dots, k$.

VD 1 : $Z = \langle B, C, D, B \rangle$ chuỗi con của $X = \langle A, \mathbf{B}, \mathbf{C}, B, \mathbf{D}, A, \mathbf{B} \rangle$ với các chỉ số $\langle 2, 3, 5, 7 \rangle$.

- Z được nói là **chuỗi con chung** của hai chuỗi X và Y nếu Z là chuỗi con của chuỗi X và Y.

VD 2 : $X = \langle A, \mathbf{B}, \mathbf{C}, B, D, \mathbf{A}, B \rangle$ và $Y = \langle \mathbf{B}, D, \mathbf{C}, \mathbf{A}, B, A \rangle, Z = \langle B, C, A \rangle$.

- $X = \langle x_1, x_2, \dots, x_m \rangle$, định nghĩa $X_i = \langle x_1, x_2, \dots, x_i \rangle$.

VD 3 : $X = \langle A, B, C, B, D, A, B \rangle, X_4 = \langle A, B, C, B \rangle$ và X_0 chuỗi rỗng.

- Bài toán : Cho hai chuỗi X và Y . Tìm chuỗi con chung dài nhất.

VD : $X = \langle A, B, C, B, D, A, B \rangle$ và $Y = \langle B, D, C, A, B, A \rangle$, $Z = \langle B, C, A \rangle$ không phải là chuỗi con chung dài nhất. Chuỗi con chung dài nhất là $\langle B, D, A, B \rangle$.

- Mệnh đề :

Cho hai chuỗi $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$, và chuỗi $Z = \langle z_1, z_2, \dots, z_k \rangle$ là *chuỗi con chung dài nhất*, khi ấy :

1. Nếu $x_m = y_n$, thì $z_k = x_m = y_n$ và Z_{k-1} một chuỗi con chung dài nhất của X_{m-1} and Y_{n-1} .
2. Nếu $x_m \neq y_n$, thì nếu $z_k \neq x_m$ thì Z một chuỗi con chung dài nhất của X_{m-1} và Y .
3. Nếu $x_m \neq y_n$, thì nếu $z_k \neq y_n$ thì Z một chuỗi con chung dài nhất của X và Y_{n-1} .

- Giải bài toán : Cho hai chuỗi X và Y. Tìm chuỗi con chung dài nhất.

Bước 1 : Tìm cấu trúc của *chuỗi con chung dài nhất*. *Tìm cấu trúc của lời giải tối ưu*.

- Gọi $c[i][j]$ là chiều dài (số ký tự) của chuỗi con chung dài nhất , chuỗi Z, của 2 chuỗi X_i và Y_j . Cần tính $c[m, n]$ và lưu các ký tự của Z.
- Theo mệnh đề ta thực hiện :
 - Nếu $x_m = y_n$, thì tính $c[m-1][n-1]$, $c[m][n] = c[m-1][n-1] + 1$,
 - Nếu $x_m \neq y_n$, thì $c[m][n] = \max \{c[m-1][n] , c[m][n-1] \}$,

Bước 2 : Phát biểu $c[m][n]$ dưới dạng đệ qui.

- Nếu $i = 0$ hay $j = 0$, thì $c[i][j] = 0$,
- Nếu $x[i] = y[j]$, thì $c[i][j] = c[i-1][j-1]$, $b[i][j] = \text{"LU"}$,
- Nếu $c[i-1][j] \geq c[i][j-1]$, thì $c[i][j] = c[i-1][j]$, $b[i][j] = \text{"up"}$ } $i, j > 0, x[i] \neq y[j]$
 Ngược lại $c[i][j] = c[i][j-1]$, $b[i][j] = \text{"left"}$.

▪ **Bước 3 :** Thuật toán.

- Input : $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$, $x[i]$, $y[i]$ ký tự thứ i của X , Y
- Output : $c[0..m, 0..n]$, $b[1..m, 1..n]$ ($b[i][j]$, $i=1, 2, \dots, m$, $j=1, 2, \dots, n$, lưu *dấu vết*.)

LCS-LENGTH($X, Y, c[][]$, $b[][]$)

```
{
1.  for (i=1 ; i <= m ; i++) c[i][0] = 0;
2.  for (j = 0 ; j <= n ; j++) c[0][j] = 0;
3.  for (i = 1 ; i <= m; i++)
4.      for ( j = 1 ; j <= n ; j++)
5.          if (x[i] == y[j]) {
6.              c[i][j] = c[i - 1][j - 1] + 1;
7.              b[i, j] = "LU" ;
            }
8.      else if (c[i - 1][j] >= c[i][j - 1]) {
9.          c[i][j] = c[i - 1][j];
10.         b[i][j] = "up" ;
            }
11.     else { c[i][j] = c[i][j - 1];
12.         b[i][j] = "left" ;
            }
}
```

▪ **Bước 3 :** Thuật toán.

PRINT-LCS(b, X, i, j)

```
{
1.  if (i = 0 or j = 0) return ;
2.  if (b[i][j] == “LU”) {
3.      PRINT-LCS(b, X, i - 1, j - 1);
4.      printf (x[i]); // ký tự thứ i của X
    }
5.  else if (b[i][j] == “up”) PRINT-LCS(b, X, i - 1, j);
6.      else PRINT-LCS(b, X, i, j - 1);
}
```

```
main()
{
    Nhập 2 chuỗi X, Y;
    LCS-LENGTH(X, Y , c, b);
    PRINT-LCS(b, X, m, n);
}
```

VD : $X = \langle A, B, C, B, D, A, B \rangle$, $Y = \langle B, D, C, A, B, A \rangle$.

	1	2	3	4	5	6	7
X =	A	B	C	B	D	A	B

	1	2	3	4	5	6
Y =	B	D	C	A	B	A

	C		0	1	2	3	4	5	6	b
				B	D	C	A	B	A	
0		0	0	0	0	0	0	0	0	
1	A	0								
2	B	0								
3	C	0								
4	B	0								
5	D	0								
6	A	0								
7	B	0								

		1	2	3	4	5	6
		B	D	C	A	B	A
1	A						
2	B						
3	C						
4	B						
5	D						
6	A						
7	B						

VD : $X = \langle A, \mathbf{B}, \mathbf{C}, \mathbf{B}, D, \mathbf{A}, B \rangle$, $Y = \langle \mathbf{B}, D, \mathbf{C}, A, \mathbf{B}, \mathbf{A} \rangle$

Hình bên dưới c và b được viết chung. Ký hiệu LU là $\nwarrow$, left là $\leftarrow$, up là $\uparrow$. .Từ ô (7, 6) đi theo các ô tô đậm. Các ký tự có ký kiệu $\nwarrow$ (LU) là ký tự trong chuỗi kết quả.

Kết quả : $\langle B, C, B, A \rangle$

		<i>j</i>	0	1	2	3	4	5	6
		<i>i</i>	<i>y_j</i>	B	D	C	A	B	A
0	<i>x_i</i>		0	0	0	0	0	0	0
1	A		0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\nwarrow$ 1	$\leftarrow$ -1	$\nwarrow$ 1
2	B		0	$\nwarrow$ 1	$\leftarrow$ -1	$\leftarrow$ -1	$\uparrow$ 1	$\nwarrow$ 2	$\leftarrow$ -2
3	C		0	$\uparrow$ 1	$\uparrow$ 1	$\nwarrow$ 2	$\leftarrow$ -2	$\uparrow$ 2	$\uparrow$ 2
4	B		0	$\nwarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\leftarrow$ -3
5	D		0	$\uparrow$ 1	$\nwarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\uparrow$ 3
6	A		0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\uparrow$ 3	$\nwarrow$ 4
7	B		0	$\nwarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\nwarrow$ 4	$\nwarrow$ 4